

Semin Bae

Tony Mione

CSE320 / Fall 2024

Nov/28/2024

Report_HW7

Summarizing everything such as each step or functions that you implemented

First Issue: Race Condition

- The first issue in the original code was a race condition occurring during the access and modification of the `num_waiting` variable. Specifically, after a customer received a haircut, the code did not decrement `num_waiting`, causing inaccuracies in tracking the number of waiting customers. This lack of synchronization could lead to multiple threads attempting to modify `num_waiting` simultaneously, resulting in data inconsistency.

Second Issue: Simultaneous Customer Arrival (may lead to deadlock)

- The second problem occurred when two customers arrived at the same time, both attempting to sit in the last available chair. The original code did not fully protect access to the waiting room seats. This could cause synchronization issues where both customers might sit in the same chair or both be turned away, leading to an inconsistent state.

Solved by

1. Introduced a `customers_served` variable to track the total number of customers who have been served or left without service.

Reason: This ensures that the barber thread knows when all customers have been processed and can terminate gracefully, preventing potential starvation or deadlock.

2. Decrement `num_waiting` after a customer finishes their haircut.

```
—  
// Added: Decrease the number of waiting customers after getting a haircut  
sem_wait(&access_seats_sem);  
num_waiting--;  
sem_post(&access_seats_sem);  
—
```

Reason: This maintains an accurate count of waiting customers and prevents race conditions by synchronizing access using `access_seats_sem`.

3. Used an additional semaphore `barber_sem` to synchronize the barber and customer threads.

- Barber Thread: Signals readiness using `sem_post(&barber_sem)`.
- Customer Thread: Waits for the barber to be ready using `sem_wait(&barber_sem)`.

Reason: This prevents scenarios where the barber might be asleep while customers are waiting or customers attempting to get a haircut when the barber is not ready, thus enhancing synchronization.

4. Protected all accesses to `num_waiting` with `access_seats_sem` semaphore.

Reason: Ensures that only one thread can modify `num_waiting` at a time, preventing race conditions when customers check and take seats or when the barber updates the count.

Challenges faced

- When incrementing the shared variable during execution, I encountered continuous race conditions, which caused the shared variable's value not to produce the desired results. This made it difficult at first to achieve correct program behavior. By using semaphores to set

conditions on the shared variable, I was able to safely modify its value until completion, ensuring that race conditions no longer occurred.

What I've learned

- I learned that when incrementing a shared variable, it's not simply a matter of increasing it with `++`. Instead, it requires following several procedures to ensure synchronization and data integrity. Proper synchronization mechanisms, like semaphores, are necessary to safely manage access to shared resources in a multithreaded environment.